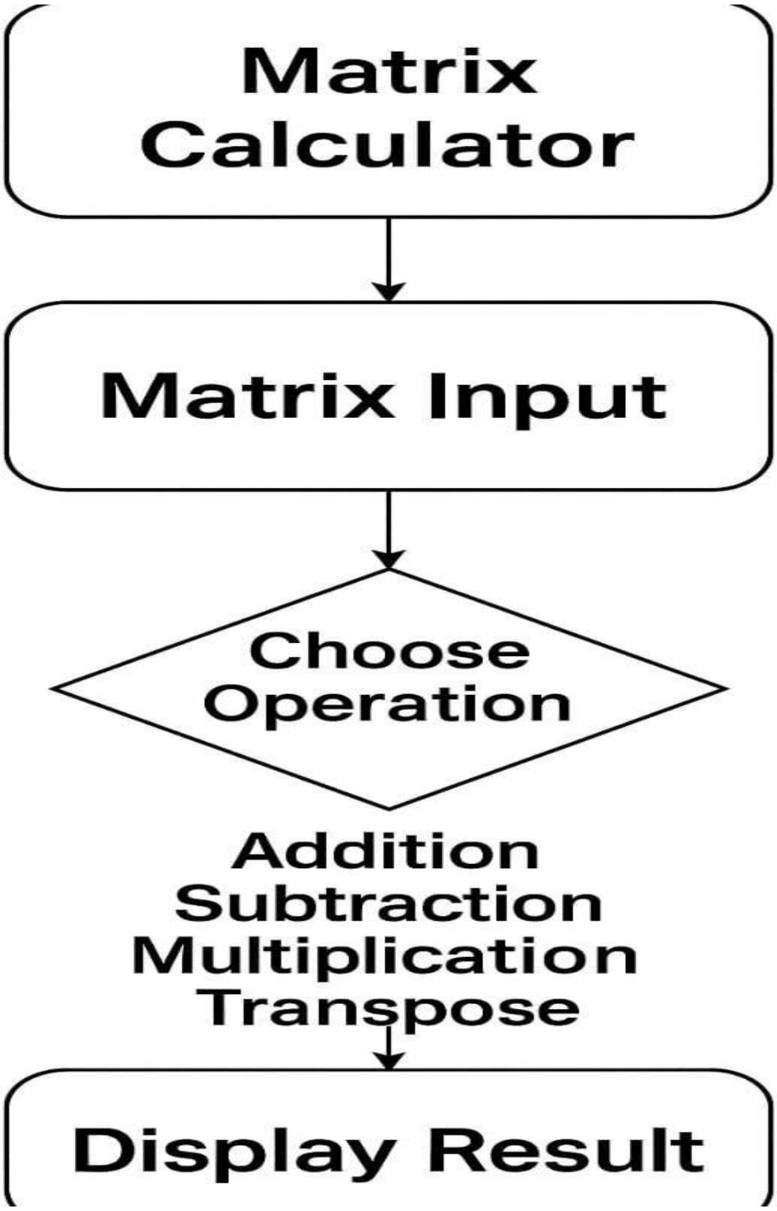
October 6 Technological University	**Faculty of Industry and Energy Technological**
Information Technology Program	**Course: Data Structures and Algorithms**
Project Name: Matrix Calculator	
1- **AHMED MOHAMED GODA — 23/121504** 2- **MANAR HASSAN GALAL — 23/121549** 3- **MARIAM WALEED ABD ELSALAM — 23/121545** 4- **ROWIDA ATEF AHMED — 23/121514** 5- **YASMIN MOSTAFA EL SAYED — 23/121558** 6- **MOHAMED YASSER KAMAL — 23/121542**	
Supervising Professor Name: Dr \Sherif Elshourbagi	
Assistant Teaching Staff: Eng \Yomna	
Second Semester: Academic Year 2024-2025	

Project Idea

The project idea revolves around designing and developing a program in C++ that performs various mathematical operations on arrays. The user can input two or more arrays, then choose the desired operation, such as addition, subtraction, multiplication, or transpse. The reslt will be displayed immediately in an organized

nd easy-to-read format.



main.cpp

```cpp
#include <iostream>
using namespace std;

class Matrix {
private:
    int mat[10][10];
    int rows, cols;

public:
    void input() {
        cout << "Enter number of rows: ";
        cin >> rows;
        cout << "Enter number of columns: ";
        cin >> cols;
        cout << "Enter matrix elements:\n";
        for (int i = 0; i < rows; i++)
            for (int j = 0; j < cols; j++) {
                cout << "Element [" << i << "][" << j << "]: ";
                cin >> mat[i][j];
            }
    }

    void display() const {
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++)
                cout << mat[i][j] << "\t";
            cout << endl;
        }
    }

    Matrix add(const Matrix &m) const {
        Matrix result;
        if (rows != m.rows || cols != m.cols) {
            cout << "Error: Dimensions must match.\n";
            result.rows = result.cols = 0;
            return result;
        }
        result.rows = rows;
```

main.cpp

```cpp
        result.rows = rows;
        result.cols = cols;
        for (int i = 0; i < rows; i++)
            for (int j = 0; j < cols; j++)
                result.mat[i][j] = mat[i][j] + m.mat[i][j];
        return result;
    }

    int getRows() const { return rows; }
    int getCols() const { return cols; }
};

int main() {
    int numMatrices;
    do {
        cout << "Enter the number of matrices to operate: ";
        cin >> numMatrices;
        if (numMatrices < 2)
            cout << "Please enter more than 1 matrix to perform the operation.\n";
    } while (numMatrices < 2);

    Matrix matrices[10];
    for (int i = 0; i < numMatrices; i++) {
        cout << "\nMatrix " << i + 1 << ":\n";
        matrices[i].input();
    }

    Matrix result = matrices[0];
    for (int i = 1; i < numMatrices; i++)
        result = result.add(matrices[i]);

    cout << "\nResult:\n";
    result.display();

    return 0;
}
```

1. Business Requirements:

To provide a matrix calculator that performs basic operations (addition, subtraction, multiplication, transpose) for educational or research purposes.

2.Stakeholder Requirements:

Students need a simple tool to perform matrix operations.

Teachers want a clear code example to explain matrix concepts in programming.

---

3. Solution Requirements:

The program must accept matrix inputs from the user.

It must perform the selected operation and display the resultMust support multiple

operations like addition, subtraction, multiplication, and transpose

4.Outputs Requirements:

The program should display the resulting matrix after the operation is performed.

5. Inputs Requirements:

Number of rows and columns for each matrix.

Matrix elements entered by the user.

7. Processes Requirements:

Check if the matrices have matching dimensions for addition or subtraction.

Ensure the first matrix's columns equal the second matrix's rows for multiplication.

Perform the selected operation and return a new matrix.

8. Performance Requirements:

Supports matrix sizes up to 10x10.

Should provide instant results after the user input.

Class Definition:

```
class Matrix {
private:
    int rows, cols;
```

A fixed-size 2D array mat[10][10] stores matrix elements.

rows and cols store the matrix dimensions.

---

2. input() Function:

```
void input() {
    cin >> rows >> cols;
    ...
```

}

Prompts the user to enter the number of rows and columns.

Takes each element of the matrix using nested for loops.

---

3. display() Function:

```
void display() {
    ...
}
```

Prints the matrix elements in a tabular format.

---

4. add(Matrix m) Function:

```
Matrix add(Matrix m) {
}
```

Checks if both matrices have the same dimensions.

Adds corresponding elements and stores them in a result matrix.

7. Processes Requirements:

Check if the matrices have matching dimensions for addition or subtraction.

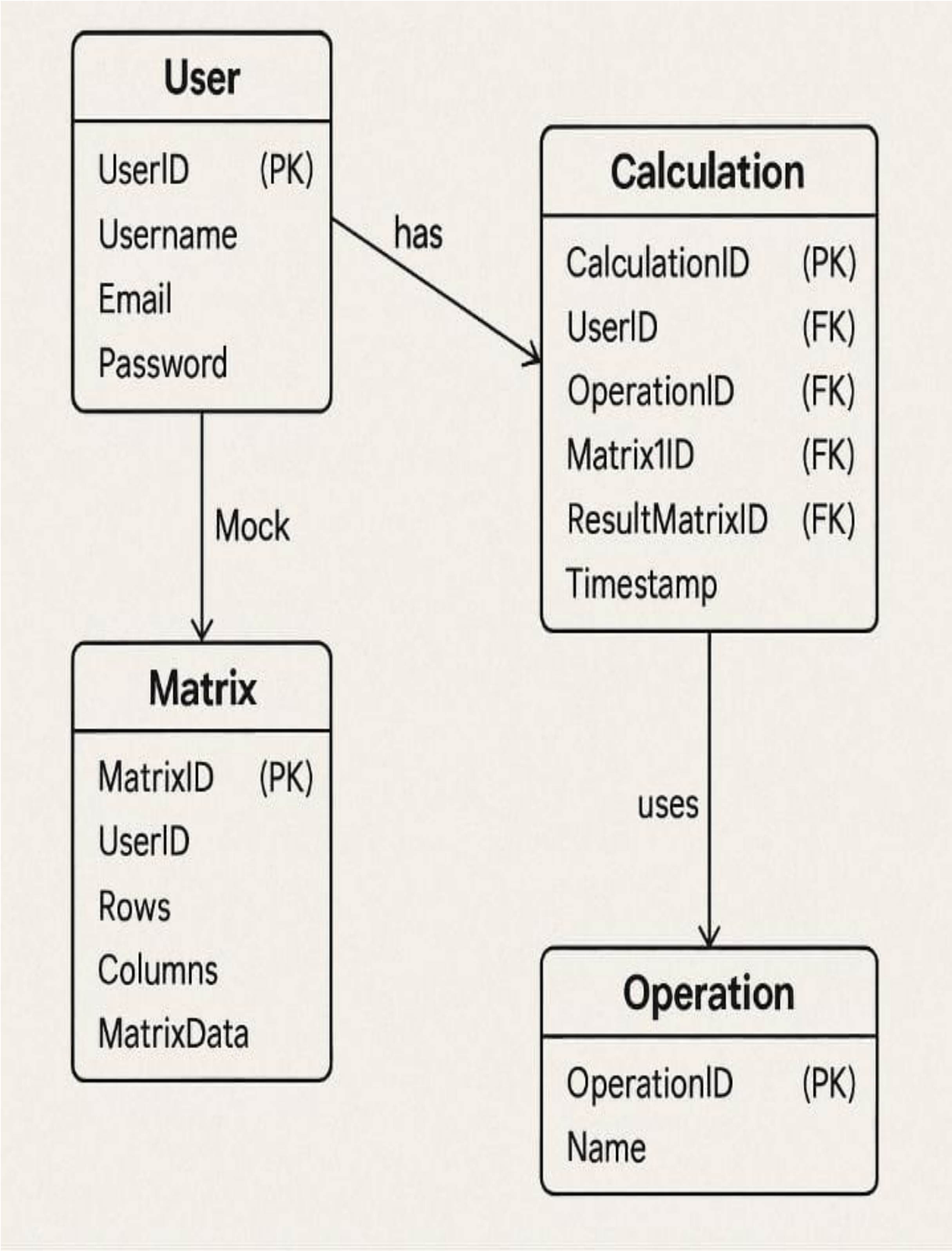
Ensure the first matrix's columns equal the second matrix's rows for multiplication.

Perform the selected operation and return a new matrix.

8. Performance Requirements:

Supports matrix sizes up to 10x10.

Should provide instant results after the user input.


User
UserID (PK)
Username
Email
Password
has
Calculation
CalculationID (PK)
UserID (FK)
OperationID (FK)
Matrix1ID (FK)
ResultMatrixID (FK)
Timestamp
Mock
Matrix
MatrixID (PK)
UserID
Rows
Columns
MatrixData
uses
Operation
OperationID (PK)
Name